

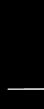
<Minimalismo Web />

Wired

Soberanía en el Historial de Proyectos

Portfolio autoalojado con `Astro`, `Turso` & `TypeScript`

```
pnpm install --future
```



Estado de la web

Ante la tendencia moderna a desarrollar webs completamente cargadas de animaciones con un rendimiento completamente desmedido he decidido intentar minimizar esto en todos mis proyectos.

Vivimos en una era de bloatware digital. Los portfolios modernos a menudo sacrifican la eficiencia por efectos visuales innecesarios.

- Cargas lentas por exceso de JS.
- Dependencia total de plataformas SaaS.
- Datos fragmentados y sin control.

CRISIS DE RENDIMIENTO

SPA Típica

850kb

Wired

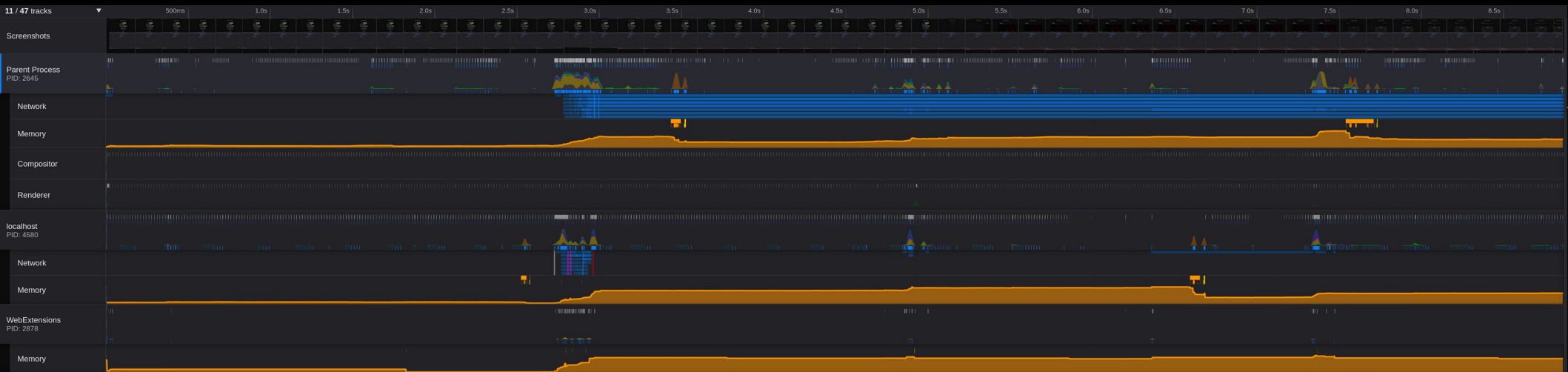
54kb

El 90% de los sitios web envían JavaScript que el usuario nunca ejecutará.

Un sistema de autoalojamiento que centraliza el historial de desarrollo, permite feedback directo y mantiene una huella digital mínima.

Principios:

1. Cero JS por defecto.
2. Datos en el Edge.
3. Tipado estricto.



Call Tree

Flame Graph

Stack Chart

Marker Chart

Marker Table

Network

All frames

Script

Native

Invert call stack

Include idle samples

Filter stacks: Enter filter terms

Complete "Parent Process"

Total (samples)	Self	
32%	194	14
22%	133	—
22%	133	133
5.8%	36	8
0.8%	5	—
0.7%	4	—
0.2%	1	—
0.2%	1	—
28%	172	5
13%	78	2
9.0%	55	—
7.2%	44	—
4.4%	27	8
3.9%	24	12
2.3%	14	—
0.2%	1	—
0.2%	1	—

RefreshDriver tick

Update the rendering Paint

Paint

Update the rendering Layout, content-visibility and resize observers

Update the rendering Animation and video frame callbacks

Update the rendering Update intersection observations

Styles

Update the rendering Update animations and send events

XPCWrappedJS method call

promise callback

Incremental GC

setTimeout handler

JSActor message handler

Incremental GC

EventListener.handleEvent

Styles

ChromeUtils::IdleDispatch handler

Paint

Call node details

Traced running time

Traced self time

Running samples

Self samples

Categories

Running sample count

Graphics

Other

Profiler

Categories

Self sample count

Graphics

Other

Profiler

Objetivos Estratégicos



Visual

Exponer proyectos con una estética limpia y funcional.



Historial

Conexión directa a base de datos para registrar progreso.



Feedback

Canal de comunicación bidireccional con visitantes.

Filosofía

Minimalismo como
disciplina en el
desarrollo de
software

Más que una Estética

Eficiencia Energética

Menos computación significa menor consumo de recursos.

Reducción Cognitiva

El diseño minimalista guía al usuario al contenido real sin distracciones.

Mantenibilidad

Menos dependencias = Menos vulnerabilidades y errores de actualización.

CERO JS por defecto

Utilizando Astro, el HTML se genera en el servidor. El cliente recibe contenido estático puro a menos que se especifique lo contrario.



```
<Component client:load />  
// Solo cuando es vital
```

PRIORIDAD DEL CONTENIDO

Tipografía

Fuentes del sistema o ligeras
para evitar FOUT.

Espaciado

Uso estratégico del
whitespace para dar aire al
código.

Accesibilidad

Contraste AA/AAA
garantizado por diseño.

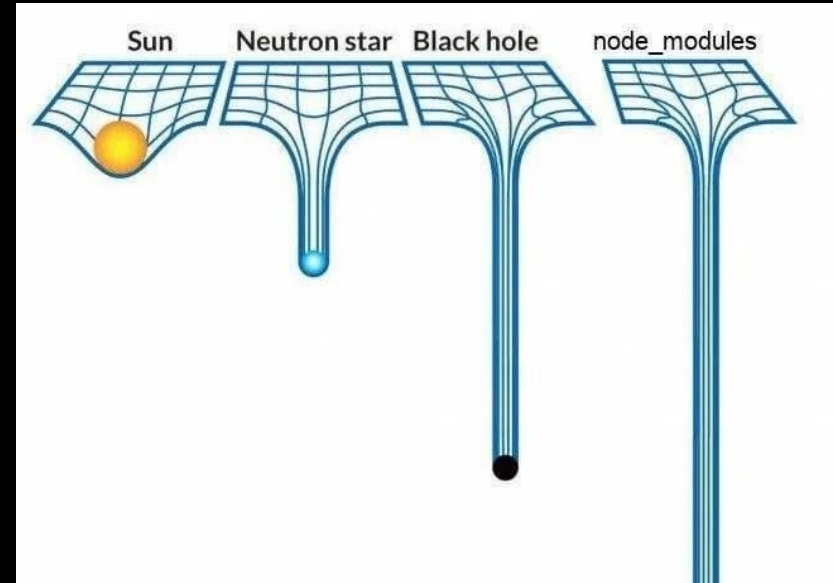
Tecnologías

Potencia sin
Compromisos

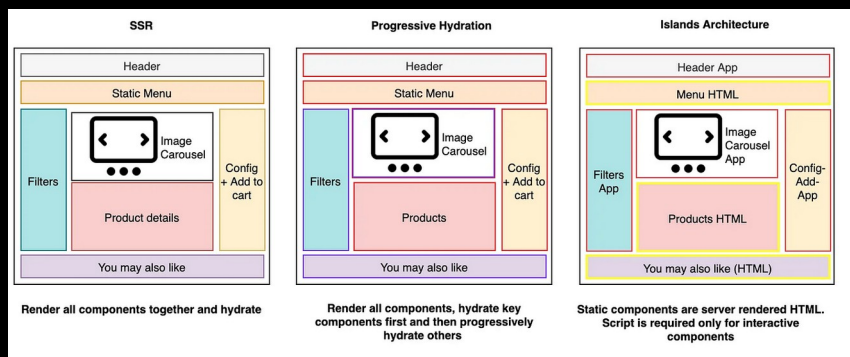
PNPM: Eficiencia en Disco

A diferencia de npm o yarn, pnpm utiliza un sistema de almacenamiento direccionado por contenido.

- Ahorro de espacio masivo.
- Instalaciones hasta 3x más rápidas.
- Estricto con las dependencias fantasma.



Astro: Islas de Interactividad



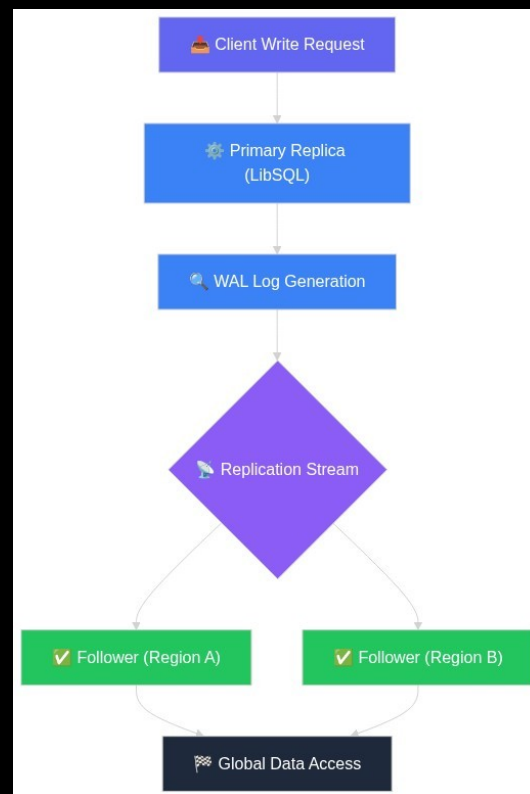
Astro permite mezclar componentes de cualquier framework (React, Vue, Svelte) pero solo envía el JS necesario al navegador.

Perfecto para un portfolio donde la mayoría del contenido es informativo.

Turso: SQLite en la Nube

`libSQL` es el fork abierto de SQLite que potencia a Turso.

- Latencia ultra-baja.
- Replicación global.
- Ideal para despliegues Serverless/Edge.



TypeScript: Tipado Robusto

El uso de TypeScript garantiza que la comunicación entre la base de datos (Turso) y el frontend (Astro) sea coherente.

```
interface Project {  
  id: string;  
  title: string;  
  status: 'active' | 'archived';  
}
```

Arquitectura

**Diseño del
Sistema**

ESQUEMA DE BASE DE DATOS

Tabla	Campos Clave	Propósito
proyectos	id, descrpcion, stack	Repositorio de proyectos
historia	id, descripcion, fecha	Registro de actividades
feedback	email, nombre, status	Retroalimentación

Bucle de Retroalimentación

Captura

Formularios ligeros con validación en el servidor.

Moderación

Estado de revisión para evitar spam.

Visualización

Comentarios filtrados por relevancia.

Implementación

Utilizamos el cliente oficial de @libsql/client para consultas asíncronas seguras.

```
try {  
  const result = await  
turso.execute('SELECT id, texto, fecha  
FROM historial ORDER BY fecha DESC');  
  
  acciones = result.rows.map(row => ({  
    id: Number(row.id),  
    texto: String(row.texto),  
    fecha: String(row.fecha)  
  }));  
  
} catch (err) {  
  console.error("Fallo al recuperar el  
historial de Turso:", err);  
  
  errorDb = "Error interno de la base de  
datos.";  
  
};
```

Ventajas de TailwindCSS

- Ahorro de clases CSS
- Evitas crear cientos de clases con nombres distintos para estilos parecidos.
- No tienes que saltar constantemente entre HTML y CSS.
- En producción, Tailwind elimina las clases que no utilizas.
- Espaciados, colores y tamaños siguen una escala común.

Estética Funcional

CSS Nativo

- Variables CSS para personalización sin runtime.

Utilidades

- Tailwind CSS (opcional) configurado para purgar el 99% del código sobrante.

Conclusión

Este proyecto demuestra que no necesitamos frameworks pesados para construir herramientas colaborativas potentes.

Sostenibilidad: Bajo consumo de CPU/Ancho de banda.

Escalabilidad: Turso permite crecer sin fricción.

Soberanía: Tus datos, tu servidor, tus reglas.

Inspirado en:

- Suckless.org
- Minthril.js
- Diferentes proyectos GNU-Linux como: Gentoo

Ideas de implementacion

- A futuro me gustaria implementar totalmente la integridad del codigo de los repositorios pudiendo ser clonados, etc.

¿Preguntas?

Gracias por
su tiempo.

exit(0)